

第 11 讲: Scaling laws 进阶与实践

CS336 Spring 2026 public videos and official course materials

目录

1 本章导读	4
1.1 本章知识路线	4
1.2 结构图	4
2 本章主线	4
2.1 本章要解决的核心问题	4
2.2 从机制到资源账本	5
2.3 从课堂讲解到可复现实验	5
3 术语、符号与问题边界	6
3.1 术语之间的关系	6
4 更复杂的 scaling 问题	6
4.1 为什么要讨论更复杂的 scaling 问题	6
4.2 更复杂的 scaling 问题的机制	7
4.3 更复杂的 scaling 问题的数学表达	7
4.4 从更复杂的 scaling 问题到程序	7
4.5 边界条件与常见误解	7
4.6 与本章其他概念的关系	7
4.7 怎样检验更复杂的 scaling 问题	8
4.8 概念辨析: 更复杂的 scaling 问题	8
4.9 更复杂的 scaling 问题的小结	8
5 数据受限与重复数据	8
5.1 为什么要讨论数据受限与重复数据	8
5.2 数据受限与重复数据的机制	9
5.3 数据受限与重复数据的数学表达	9
5.4 从数据受限与重复数据到程序	9
5.5 边界条件与常见误解	9

5.6	与本章其他概念的关系	9
5.7	怎样检验数据受限与重复数据	9
5.8	概念辨析：数据受限与重复数据	10
5.9	数据受限与重复数据的小结	10
6	从 loss 到能力指标	10
6.1	为什么要讨论从 loss 到能力指标	10
6.2	从 loss 到能力指标的机制	10
6.3	从 loss 到能力指标的数学表达	11
6.4	从从 loss 到能力指标到程序	11
6.5	边界条件与常见误解	11
6.6	与本章其他概念的关系	11
6.7	怎样检验从 loss 到能力指标	11
6.8	概念辨析：从 loss 到能力指标	12
6.9	从 loss 到能力指标的小结	12
7	综合讨论：从局部机制到完整系统	12
7.1	更复杂的 scaling 问题在系统中的位置	12
7.2	数据受限与重复数据在系统中的位置	13
7.3	从 loss 到能力指标在系统中的位置	13
7.4	把本章知识用于读论文和复现实验	13
8	例题：把概念变成可计算检查	14
8.1	为什么数据 pipeline 改变后要重拟合 scaling law	14
9	实验设计：从小例子到可复现结论	14
9.1	最小输入	14
9.2	资源账本	14
9.3	单变量消融	14
9.4	失败条件	14
9.5	记录与报告	14
9.6	从小实验到大结论	15
9.7	把本章重新读成一条证据链	15
10	课程资料与回看路径	15
10.1	视频回看路径	15
10.2	课程资料阅读路径	16
11	实践说明、历史位置与风险	16

12 复习要点	17
12.1 综合自测	17
13 总结与延伸	17
13.1 本章小结	17
13.2 延伸解释	17
13.3 练习	18

1 本章导读

本章讨论 **Scaling laws 进阶与实践**。CS336 的第一层目标不是把现成大模型当作黑箱使用，而是把 language model 从输入文本、训练目标、模型结构、数据、系统和评估这些部件重新拆开。只有拆开之后，读者才能判断一个设计选择究竟改变了什么。设想基础 scaling law 给出一条漂亮曲线，但你的数据有限、训练 schedule 改了、benchmark 也不只看 loss。进阶 scaling 的问题是：哪些外推仍可信，哪些必须重新校准？本章对应 CS336 Spring 2026 第 11 个公开视频，并结合课程页提供的可解析资料。视频和课程页给出事实边界；正文把这些材料改写为可自学的教材章节。阅读本章时，先理解问题为什么存在，再看定义、公式和实现形态，随后通过例题和练习检查自己是否能独立复现这条 reasoning chain。

Lecture 11

SCALING – CASE STUDY AND DETAILS

CS336



图 1: 第 11 讲的课程图像锚点。

1.1 本章知识路线

- 本章首先说明 Scaling laws 进阶与实践为什么是 language modeling pipeline 中不可跳过的一环。
- 其次定义本章反复使用的术语，并说明这些术语对应的计算对象或系统对象。
- 随后用公式和伪代码表达主要机制，让概念能够落到可计算的变量上。
- 最后给出例题、风险讨论和练习，便于读者检查自己是否真正掌握了机制。

1.2 结构图

下面的图用于帮助读者定位本章的核心结构。先看概念之间的依赖，再回到正文中的公式、伪代码和例题。

2 本章主线

Scaling laws 进阶与实践的主线是从小规模实验走向大规模决策。读者需要同时追踪参数量、token 数、compute budget、loss 和 benchmark 之间的关系。

2.1 本章要解决的核心问题

本讲继续 scaling laws，重点转向实践细节：muP、WSD learning rate、batch/LR scaling、MiniCPM 和 DeepSeek 等公开案例。读完这一段后，应能用一两句话说明 Scaling laws 进阶与实践要解决的瓶颈，并指出

Motivation today

What is the best practices for scaling and hparam tuning LMs?

- Does chinchilla's approach to scaling actually work?
- Can we save compute when training and fitting these things?
- Should we be picking particular architectures / parametrizations to scale nicely?

图 2: 官方材料图页: Scaling laws 进阶与实践的课程页/PDF/script 图像锚点。

Lecture 11

SCALING – CASE STUDY AND DETAILS

CS336

图 3: 官方材料图页: Scaling laws 进阶与实践的课程页/PDF/script 图像锚点。

这个瓶颈发生在数据、模型、优化、系统还是评估层。

2.2 从机制到资源账本

进阶 scaling 的难点是如何降低拟合 scaling law 本身的成本。WSD 等 schedule 允许从稳定阶段重启 decay, 以便更便宜地估计数据-模型 tradeoff。随后把机制写成账本: 输入规模是什么, 主要状态是什么, 成本或质量指标怎样随变量改变。这个账本让后面的公式不再只是符号。

2.3 从课堂讲解到可复现实验

实践中要同时预测 loss、选择 batch、学习率、模型宽深比和训练 tokens; 任何 recipe 改变都需要重新验证外推可靠性。最后把课堂讲解转成可复现实验: 固定协议, 改变一个变量, 记录中间量, 并解释结果为何支持或反驳原来的机制判断。

3 术语、符号与问题边界

本章的术语横跨模型、数据和系统。读者需要先知道每个词指向的对象：有些词描述数学目标，有些词描述张量形状，有些词描述硬件瓶颈，还有一些词描述评估协议。术语边界不清，后面的公式和实现就会变成空洞的缩写。

#	术语	阅读要求
1	forecasting（预测）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
2	overtraining（过训练/超额训练）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
3	data-constrained scaling（数据受限缩放）	说明它如何改变训练样本或 token 分布，并给出一个会改变结论的数据边界条件。
4	muP	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
5	emergence（涌现）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。

这些术语之间有依赖关系。例如 tokenization 改变 sequence length，sequence length 又改变 attention FLOPs、KV cache 和 evaluation token budget；parallelism 改变显存占用，也改变通信瓶颈和故障恢复方式。

3.1 术语之间的关系

在本章中，forecasting（预测）给出问题入口，overtraining（过训练/超额训练）描述主要机制，data-constrained scaling（数据受限缩放）则把机制连接到可测量的结果。读教材时应当沿着这个链条追问：输入是什么，中间状态怎样改变，输出指标为什么随之变化。课程材料还会反复出现 scaling、batch、optimal、scale、learning、chinchilla 等词。它们不是一组同义标签，而是提示本章的不同层次：有的指对象，有的指操作，有的指约束或指标。因此，术语表不是背诵清单，而是后文阅读的索引。遇到一个术语时，先定位它属于 representation、optimization、systems、data 还是 evaluation，再判断它改变哪一项账本。

4 更复杂的 scaling 问题

现在进入更复杂的 scaling 问题。这一节把模型大小、训练 token、compute budget 和 loss 放在同一坐标系里，讨论何时可以从小实验推断大训练。

4.1 为什么要讨论更复杂的 scaling 问题

视频开场称本讲是 scaling journey 的继续，覆盖更高级细节和实践中扩展模型的重要问题。实践中不只问 loss 怎么随 (N,D) 变，还要问数据受限、训练长度、architecture 改变、optimizer 改变时预测是否仍成立。scaling law 的价值在于减少大 run 风险，而不是替代工程监控。更复杂的 scaling 问题把 Scaling laws 进阶与实践连接到大规模决策。接下来要看的不是曲线是否平滑，而是实验点、预算变量和外推假设是否一致。

4.2 更复杂的 scaling 问题的机制

视频开场称本讲是 scaling journey 的继续，覆盖更高级细节和实践中扩展模型的重要问题。在更复杂的 scaling 问题中，核心对象是模型规模、token 数和训练预算：小实验只有在预算、数据和优化条件清楚时，才可能支持大规模外推。实践中不只问 loss 怎么随 (N, D) 变，还要问数据受限、训练长度、architecture 改变、optimizer 改变时预测是否仍成立。因而同一个方法在不同规模下可能改变不同的变量，尤其是参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。比较方法时，必须同时给出问题规模和实验条件。scaling law 的价值在于减少大 run 风险，而不是替代工程监控。可检验性来自多个实验点、残差分析和外推误差估计；单条曲线若没有不确定性，就不能支撑大规模决策。因此，更复杂的 scaling 问题的学习目标是建立一套外推判断：读者应能说明小实验为何能或不能指导 Scaling laws 进阶与实践中的大训练配置。

4.3 更复杂的 scaling 问题的数学表达

$$L(C) = L_{\infty} + aC^{-\gamma}$$

符号说明： C 是总 compute；这个一维形式常用于简化预测，但会隐藏 N, D tradeoff。这个关系式给出更复杂的 scaling 问题的缩放假设。它把参数量、训练 token 数和 compute budget 放到同一张账本中，重点是判断小实验能否可靠外推到大训练。在 scaling 实验中，validation loss、benchmark transfer 和训练吞吐都只是不同侧面的 proxy。外推时要说明哪个 proxy 被拟合，哪个 proxy 只是辅助诊断。

4.4 从更复杂的 scaling 问题到程序

把更复杂的 scaling 问题写成程序后，抽象机制会变成输入、状态和输出之间的变换。下面的伪代码保留核心计算顺序。

```
fit_compute_curve(runs)
check_breakpoints_for_new_regime()
forecast_only_with_errorBars()
```

阅读这段流程时，重点看小规模实验怎样产生可外推曲线。参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系需要在每个实验点上同时记录，缺少其中一项都会改变 scaling 解释。

4.5 边界条件与常见误解

- 单变量 compute curve 可能掩盖模型大小和数据量的不同失败模式。
- architecture 改变后，把旧曲线直接外推很危险。
- 小规模 scaling 曲线不一定覆盖大训练的优化不稳定、数据瓶颈和系统瓶颈。
- compute-optimal 结论依赖目标指标；以 loss 最优并不自动意味着 benchmark、latency 或成本最优。

4.6 与本章其他概念的关系

更复杂的 scaling 问题连接本章的小实验和大规模决策：术语表标出预算变量，公式给出外推结构，伪代码组织实验点，例题检查 compute-optimal 判断是否稳健。

4.7 怎样检验更复杂的 scaling 问题

检验更复杂的 scaling 问题时，先把抽象说法落到模型规模、token 数和训练预算上。与 forecasting、compute、regime、loss、practice 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： $\backslash(C\backslash)$ 是总 compute；这个一维形式常用于简化预测，但会隐藏 $\backslash(N,D\backslash)$ tradeoff。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。scaling 消融通常一次只改变参数量、token 数、数据 pipeline 或优化 schedule。若多个轴同时移动，曲线拟合会混合不同机制。最后检查失败条件。工程判断往往在反例中变清楚：单变量 compute curve 可能掩盖模型大小和数据量的不同失败模式；architecture 改变后，把旧曲线直接外推很危险。复习时可以把这些限制改写成反例测试，观察它们怎样改变参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。把实验结论放回 Scaling laws 进阶与实践时，要说明更复杂的 scaling 问题改变的是预算轴、拟合曲线、外推误差还是目标指标。能拆开这些因素，才说明读者真正理解 scaling。

4.8 概念辨析：更复杂的 scaling 问题

forecasting、compute、regime、loss、practice 常一起出现在 scaling 讨论中，但它们分别指预算轴、拟合对象、外推假设或风险来源。读者应先确定曲线中的自变量和因变量。区分这些词时，可以先问它们是否改变参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕更复杂的 scaling 问题复习时，可以画三到五个小实验点：记录参数量、token 数、预算和 loss，再检查外推是否合理。

4.9 更复杂的 scaling 问题的小结

- 讨论更复杂的 scaling 问题时，应同时报告问题规模、实验设置和主要观测变量，尤其是参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。
- 如果方法声称可以外推，要说明实验点覆盖范围、拟合形式和数据管线是否一致；曲线本身不是自然定律。
- scaling 结论需要把 loss、benchmark 和成本之间的转换关系说清楚，否则 compute-optimal 只是一条局部经验曲线。

5 数据受限与重复数据

现在进入数据受限与重复数据。这一节关心训练样本如何形成、被筛选并进入混合分布；它直接决定模型看到什么，也决定 evaluation 是否可能被污染。

5.1 为什么要讨论数据受限与重复数据

当高质量 token 有限时，简单增加训练 tokens 可能意味着重复数据或低质量数据。重复训练同一数据会改变 scaling 行为：短期可能降低 loss，长期可能过拟合或降低泛化。这为后续 data lectures 铺垫：scaling law 必须和数据来源、过滤、去重、混合策略一起讨论。数据受限与重复数据把 Scaling laws 进阶与实践连接到训练分布。接下来要看的不是语料名称，而是样本来源、处理顺序和版本记录怎样改变模型可学习的证据。

5.2 数据受限与重复数据的机制

当高质量 token 有限时，简单增加训练 tokens 可能意味着重复数据或低质量数据。在数据受限与重复数据中，核心对象是样本、来源和分布状态：数据管线每改变一次样本集合，模型实际学习的分布也随之改变。重复训练同一数据会改变 scaling 行为：短期可能降低 loss，长期可能过拟合或降低泛化。因而同一个方法在不同规模下可能改变不同的变量，尤其是样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。比较方法时，必须同时给出问题规模和实验条件。这为后续 data lectures 铺垫：scaling law 必须和数据来源、过滤、去重、混合策略一起讨论。可检验性来自 provenance 记录、过滤前后计数、抽样审计和 held-out contamination 检查；没有这些记录，数据结论无法复现。因此，数据受限与重复数据的学习目标是建立一条数据谱系：读者应能从原始来源追到训练 token，并说明每个处理步骤改变了什么分布。

5.3 数据受限与重复数据的数学表达

$$D_{\text{effective}} \leq D_{\text{raw}} \quad \text{when duplicates and low-quality tokens are present}$$

符号说明： D_{raw} 是原始 token 数， $D_{\text{effective}}$ 是对泛化真正有贡献的有效 token 数。这个关系式刻画数据受限与重复数据对数据分布的影响。读者应关心哪些变量来自采集过程，哪些变量来自过滤或混合策略，哪些变量只能通过抽样审计估计。在数据实验中，去重率、过滤通过率、语言比例和抽样人工判读都是 proxy。它们不能单独代表数据质量，必须和来源记录、版本号以及污染检查一起解释。

5.4 从数据受限与重复数据到程序

把数据受限与重复数据写成程序时，要明确每一步怎样改写样本集合。下面的伪代码强调输入、过滤、统计和输出之间的关系。

```
effective_tokens = estimate_after_dedup_and_filter(raw_tokens)
refit_scaling_law_if_data_pipeline_changes()
```

阅读这段流程时，重点看数据受限与重复数据怎样改变样本集合。每个过滤、去重或混合步骤都应留下计数和抽样检查，使样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险能够被复核。

5.5 边界条件与常见误解

- 有效 token 数不可直接从文件大小读取。
- 低质量数据可能增加 token count 但恶化 downstream behavior。
- 小样本抽样只能发现部分数据问题，不能证明语料整体无污染或无偏差。
- 数据版本、许可状态和处理脚本如果缺失，后续 evaluation 与 scaling 结论都会失去可复现基础。

5.6 与本章其他概念的关系

数据受限与重复数据连接本章的数据来源和模型行为：定义说明样本如何进入语料，公式刻画分布变化，伪代码展示处理顺序，例题检验数据决策怎样影响训练结论。

5.7 怎样检验数据受限与重复数据

检验数据受限与重复数据时，先把抽象说法落到样本、来源和分布状态上。与 data、duplicates、effective tokens、overtraining、quality 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不

能得到可复现结论。一个合适的小实验应当保留本节机制，同时让样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： $(D_{\text{ext}\{\text{raw}\}})$ 是原始 token 数， $(D_{\text{ext}\{\text{effective}\}})$ 是对泛化真正有贡献的有效 token 数。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。数据消融通常一次只改变过滤阈值、去重策略、混合权重或数据版本。若同时更换 tokenizer、语料和评估集，就无法知道改进来自哪一步。最后检查失败条件。工程判断往往在反例中变清楚：有效 token 数不可直接从文件大小读取；低质量数据可能增加 token count 但恶化 downstream behavior。复习时可以把这些限制改写成反例测试，观察它们怎样改变样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。把实验结论放回 Scaling laws 进阶与实践时，要说明数据受限与重复数据改变的是样本分布、数据质量、污染风险还是可复现性。能追到这些来源，才说明读者真正理解数据机制。

5.8 概念辨析：数据受限与重复数据

data、duplicates、effective tokens、overtraining、quality 常一起出现在数据管线中，但它们分别指来源、处理动作、版本记录或风险标记。读者应先判断它们改变的是样本集合还是审计证据。其中，data（数据）是模型能力、偏差和法律风险的主要来源。区分这些词时，可以先问它们是否改变样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕数据受限与重复数据复习时，可以构造一小批文档：保留来源和版本，运行一个处理步骤，再比较样本计数和抽样质量。

5.9 数据受限与重复数据的小结

- 讨论数据受限与重复数据时，应同时报告问题规模、实验设置和主要观测变量，尤其是样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。
- 如果方法声称数据更好，要说明过滤、去重、混合或合成步骤改变了哪一部分分布，并检查 benchmark contamination。
- 数据结论需要抽样审计和版本记录；没有 provenance 的语料变化不能支持可引用结论。

6 从 loss 到能力指标

现在进入从 loss 到能力指标。这一节把模型大小、训练 token、compute budget 和 loss 放在同一坐标系里，讨论何时可以从小实验推断大训练。

6.1 为什么要讨论从 loss 到能力指标

loss 平滑、可测、适合拟合，但用户关心的是 reasoning、coding、instruction following 等能力。能力指标可能出现阈值效应或评估噪声，使其比 loss 更难预测。进阶 scaling 分析要把 validation loss、benchmark score、推理成本和安全约束一起纳入。从 loss 到能力指标把 Scaling laws 进阶与实践连接到大规模决策。接下来要看的不是曲线是否平滑，而是实验点、预算变量和外推假设是否一致。

6.2 从 loss 到能力指标的机制

loss 平滑、可测、适合拟合，但用户关心的是 reasoning、coding、instruction following 等能力。在从 loss 到能力指标中，核心对象是模型规模、token 数和训练预算：小实验只有在预算、数据和优化条件清楚时，才

可能支持大规模外推。能力指标可能出现阈值效应或评估噪声，使其比 loss 更难预测。因而同一个方法在不同规模下可能改变不同的变量，尤其是参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。比较方法时，必须同时给出问题规模和实验条件。进阶 scaling 分析要把 validation loss、benchmark score、推理成本和安全约束一起纳入。可检验性来自多个实验点、残差分析和外推误差估计；单条曲线若没有不确定性，就不能支撑大规模决策。因此，从 loss 到能力指标的学习目标是建立一套外推判断：读者应能说明小实验为何能或不能指导 Scaling laws 进阶与实践中的大训练配置。

6.3 从 loss 到能力指标的数学表达

$$\mathbb{E}[\text{score}] \approx h(L, \text{prompt}, \text{eval distribution})$$

符号说明： h 表示从 loss 到 benchmark score 的经验映射；它依赖评估任务和 prompting。这个关系式给出从 loss 到能力指标的缩放假设。它把参数量、训练 token 数和 compute budget 放到同一张账本中，重点是判断小实验能否可靠外推到大训练。在 scaling 实验中，validation loss、benchmark transfer 和训练吞吐都只是不同侧面的 proxy。外推时要说明哪个 proxy 被拟合，哪个 proxy 只是辅助诊断。

6.4 从 loss 到能力指标到程序

把从 loss 到能力指标写成程序后，抽象机制会变成输入、状态和输出之间的变换。下面的伪代码保留核心计算顺序。

```
for checkpoint in training_curve:
    measure_loss(checkpoint)
    measure_benchmarks(checkpoint)
    fit_loss_to_score_mapping()
```

阅读这段流程时，重点看小规模实验怎样产生可外推曲线。参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系需要在每个实验点上同时记录，缺少其中一项都会改变 scaling 解释。

6.5 边界条件与常见误解

- benchmark score 的方差和 contamination 会扭曲趋势。
- 能力不是只由 pretraining loss 决定，post-training 也会改变可见行为。
- 小规模 scaling 曲线不一定覆盖大训练的优化不稳定、数据瓶颈和系统瓶颈。
- compute-optimal 结论依赖目标指标；以 loss 最优并不自动意味着 benchmark、latency 或成本最优。

6.6 与本章其他概念的关系

从 loss 到能力指标连接本章的小实验和大规模决策：术语表标出预算变量，公式给出外推结构，伪代码组织实验点，例题检查 compute-optimal 判断是否稳健。

6.7 怎样检验从 loss 到能力指标

检验从 loss 到能力指标时，先把抽象说法落到模型规模、token 数和训练预算上。与 benchmark、emergence、score、loss、post-training 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让参数量、训练 token 数、compute budget、loss

外推误差和 benchmark 转化关系中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： $\backslash(h\backslash)$ 表示从 loss 到 benchmark score 的经验映射；它依赖评估任务和 prompting。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。scaling 消融通常一次只改变参数量、token 数、数据 pipeline 或优化 schedule。若多个轴同时移动，曲线拟合会混合不同机制。最后检查失败条件。工程判断往往在反例中变清楚：benchmark score 的方差和 contamination 会扭曲趋势；能力不是只由 pretraining loss 决定，post-training 也会改变可见行为。复习时可以把这些限制改写成反例测试，观察它们怎样改变参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。把实验结论放回 Scaling laws 进阶与实践时，要说明从 loss 到能力指标改变的是预算轴、拟合曲线、外推误差还是目标指标。能拆开这些因素，才说明读者真正理解 scaling。

6.8 概念辨析：从 loss 到能力指标

benchmark、emergence、score、loss、post-training 常一起出现在 scaling 讨论中，但它们分别指预算轴、拟合对象、外推假设或风险来源。读者应先确定曲线中的自变量和因变量。区分这些词时，可以先问它们是否改变参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕从 loss 到能力指标复习时，可以画三到五个小实验点：记录参数量、token 数、预算和 loss，再检查外推是否合理。

6.9 从 loss 到能力指标的小结

- 讨论从 loss 到能力指标时，应同时报告问题规模、实验设置和主要观测变量，尤其是参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。
- 如果方法声称可以外推，要说明实验点覆盖范围、拟合形式和数据管线是否一致；曲线本身不是自然定律。
- scaling 结论需要把 loss、benchmark 和成本之间的转换关系说清楚，否则 compute-optimal 只是一条局部经验曲线。

7 综合讨论：从局部机制到完整系统

本章的主题是 Scaling laws 进阶与实践，但它不应被读成几个相邻知识点的拼接。更复杂的 scaling 问题、数据受限与重复数据、从 loss 到能力指标共同回答同一个问题：语言模型系统中哪些对象可以被定义，哪些成本可以被估算，哪些结论必须通过实验或评估来确认。这些对象包括 forecasting（预测）、overtraining（过训练/超额训练）、data-constrained scaling（数据受限缩放）、muP、emergence（涌现）。读者可以把它们看成一组接口：有的接口连接文本和 token，有的连接模型结构和张量计算，有的连接数据样本和训练目标，有的连接离线评估和在线服务。接口一旦改变，后面的成本、错误类型和优化空间也会随之改变。

7.1 更复杂的 scaling 问题在系统中的位置

更复杂的 scaling 问题首先提供一个局部解释：视频开场称本讲是 scaling journey 的继续，覆盖更高级细节和实践中扩展模型的重要问题。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，更复杂的 scaling 问题会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，更复杂的 scaling 问题不能脱离边

界条件理解。一个关键 caveat 是：单变量 compute curve 可能掩盖模型大小和数据量的不同失败模式。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.2 数据受限与重复数据在系统中的位置

数据受限与重复数据首先提供一个局部解释：当高质量 token 有限时，简单增加训练 tokens 可能意味着重复数据或低质量数据。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，数据受限与重复数据会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，数据受限与重复数据不能脱离边界条件理解。一个关键 caveat 是：有效 token 数不可直接从文件大小读取。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.3 从 loss 到能力指标在系统中的位置

从 loss 到能力指标首先提供一个局部解释：loss 平滑、可测、适合拟合，但用户关心的是 reasoning、coding、instruction following 等能力。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，从 loss 到能力指标会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，从 loss 到能力指标不能脱离边界条件理解。一个关键 caveat 是：benchmark score 的方差和 contamination 会扭曲趋势。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.4 把本章知识用于读论文和复现实验

读相关论文时，可以把本章变成一个检查表。第一，论文是否清楚定义了输入规模、模型规模、数据来源和评估协议；第二，论文声称的改进落在哪个变量上；第三，作者是否报告了会让结论失效的设置；第四，实验是否能分离模型结构、数据质量、优化设置和系统实现的影响。复现实验时，也应避免直接追求大规模。先用最小程序复现一个公式、一个伪代码分支或一个 profile 现象，再扩大输入规模。这样做的原因很简单：如果小规模程序无法解释中间量，大规模训练只会把错误隐藏在日志和随机波动里。把这些检查合在一起，本章给出的不是一个固定答案，而是一种阅读 CS336 的方法：每个模型机制都要落到变量，每个变量都要落到测量，每个测量都要说明协议，每个协议都要承认边界。因此，本章中的任何一句结论都可以被改写成一个可引用的完整句式：在给定数据、模型、硬件和评估协议下，某个机制改变了某个变量，并通过某个观测指标表现出来。这样的句子比单独背诵术语更长，但它包含了自学、复现和引用时真正需要的信息。把本章用于自学时，可以按三遍阅读。第一遍只追踪对象和定义，确认每个术语到底指向文本、token、张量、样本、请求还是评估记录。第二遍追踪公式和伪代码，确认每一步改变了哪些状态。第三遍追踪实验和 caveat，确认哪些结论只在特定规模、数据或硬件条件下成立。把本章用于复习时，则应反过来操作：先从一个失败案例或异常指标出发，再回到相应机制。loss 曲线异常可能指向数据、优化或模型容量；吞吐异常可能指向 kernel、通信或调度；benchmark 异常可能指向 contamination、prompt protocol 或采样设置。能够从异常反推机制，是掌握本章的实用标准。

8 例题：把概念变成可计算检查

8.1 为什么数据 pipeline 改变后要重拟合 scaling law

同一模型大小和 token 数，换用更强过滤后的数据。

- validation loss 曲线可能整体下移，因为有效 token 质量提高。
- 不同 benchmark 的改善幅度不一定一致。
- 旧曲线的外推误差不再代表新数据分布。

结论。 scaling law 不是模型大小的自然属性，而是模型、数据、优化和评估分布的联合经验规律。这个例题的目的不是替代完整工程实现，而是给出一种最小推理形式：把问题转成变量，写出近似公式或伪代码，再说明近似何时失效。

9 实验设计：从小例子到可复现结论

学习 Scaling laws 进阶与实践不能停在概念层。一个教材化结论至少需要四件事：可构造的输入、可观测的中间量、可比较的指标，以及清楚的失败条件。下面的实验设计不是要求训练大模型，而是要求读者用小规模程序复现本章机制。

9.1 最小输入

先构造只触发本章核心机制的小输入。例如 tokenizer 章节可以用几个包含 rare words 和 Unicode 字符的字符串；GPU 章节可以用一个 elementwise kernel 和一个 GEMM；data 章节可以用十几篇近重复文档。小输入的价值在于它让错误可见。

9.2 资源账本

对同一输入写下 FLOPs、bytes moved、显存峰值、通信量或样本数量的估算。估算不需要一开始就精确，但必须说明单位和近似假设。随后用 profiler、benchmark、validation loss 或人工检查去验证估算是否同量级。

9.3 单变量消融

一次只改变一个因素，例如 vocabulary size、head 数、batch size、filter threshold、reward scale、decoding temperature 或 GPU 数量。若多个变量同时变化，实验只能说明系统变了，不能说明机制是什么。

9.4 失败条件

最后要刻意触发失败：极端 context length、错误 dtype、过小 batch、污染 benchmark、低质量数据或不可靠 verifier。失败条件常常比成功曲线更能说明一个方法的边界。

9.5 记录与报告

实验记录应能让另一个读者复现同一结论。最少要写清楚输入构造、代码版本、随机种子、硬件、batch shape、数据版本、评价指标和异常处理方式。报告结论时不要只写“更快”或“更好”，而要说明快在哪里、好

在哪个指标上，以及是否牺牲了内存、稳定性、数据质量或可解释性。常见报告错误是把局部现象写成普遍规律。例如一次小模型实验里的 loss 改善，不等于更大模型或不同数据分布上也会改善；一次 kernel benchmark 的吞吐提升，不等于端到端 serving latency 一定降低。教材中的实验报告应始终把结论限定在可复现条件内。

9.6 从小实验到大结论

小实验的作用不是替代课程中的完整训练或系统实现，而是建立一条可检查的推理链。先在小输入上确认变量方向，再在中等规模上确认数量级，最后才讨论是否能外推到更大模型、更多 token 或更复杂 workload。缺少中间层级时，大结论通常只是在复述直觉。因此，实验报告最好同时包含正例和反例：正例说明机制在受控条件下怎样工作，反例说明条件稍变时哪里失效。对 CS336 这样的课程，反例尤其重要，因为 language model 的错误常来自多个层面叠加：数据分布、优化稳定性、硬件瓶颈、评估协议和服务负载都可能同时改变观察结果。还要记录资料版本与复现边界。公开视频、课程页、配套代码和 PDF 可能在不同时间更新；复现实验应写明使用的材料版本，并说明哪些结论来自课程事实，哪些是为了帮助理解而加入的解释性推导。这样才能把学习笔记变成可检查的技术记录。若复现实验与课程结论不一致，首先检查输入、版本和评估协议，而不是立即修改模型假设。

9.7 把本章重新读成一条证据链

完成小实验之后，可以把 Scaling laws 进阶与实践重新读成一条证据链：先用“更复杂的 scaling 问题”确定问题入口，再用“数据受限与重复数据”解释主要机制，最后用“从 loss 到能力指标”检查边界或外推条件。这种读法比按页背诵更接近教材训练，因为它要求每个结论都能说明前提、变量和证据。本章反复使用的术语包括 forecasting（预测）、overtraining（过训练/超额训练）、data-constrained scaling（数据受限缩放）、muP、emergence（涌现）。复习时不要只写定义，而要为每个术语补上一句操作性说明：它在课程代码、公式、数据记录或评估协议中对应什么对象；如果这个对象被删除、替换或缩放，哪一个观测量会变化。最后，用一个反例结束复习。反例可以是资源估算不准、数据污染、reward 失真、benchmark 解析失败、长上下文显存爆炸或 scaling 外推失效。能解释反例的章节，才真正具备自学和引用价值；不能解释反例的章节，只是把课程材料换了一种排版。引用本章结论时，还应保留来源边界：哪些说法直接来自视频、课程页、slides 或代码，哪些是为了串联教材叙述而加入的解释性推导。这个边界不会削弱结论，反而让读者知道何处可以复核，何处需要在自己的实验中重新验证。如果后续课程材料更新，也可以沿着同一证据链替换来源，而不必重写整章结构。

10 课程资料与回看路径

教材正文已经把主要概念、公式和实现逻辑组织成连续叙述；本节只提供回到课程材料的阅读路径。读者复习时可以先完成前文练习，再按下面的时间段或资料组核对细节。

10.1 视频回看路径

时间	关键词	复习任务
00:00:05-00:06:25	going, talk, scale, scaling, think, we're, learning	回看定义、例子、推导或 caveat 在该段中的位置。
00:19:14-00:26:00	scaling, they're, then, going, sparsity, learning, different	回看定义、例子、推导或 caveat 在该段中的位置。
00:39:25-00:45:53	think, maybe, scaling, then, scale., scale, over	回看定义、例子、推导或 caveat 在该段中的位置。

00:51:54-00:57:45	muon, think, then, it's, scale, matrix, very	回看定义、例子、推导或 caveat 在该段中的位置。
01:10:16-01:16:41	going, scaling, learning, very, it's, over, different	回看定义、例子、推导或 caveat 在该段中的位置。

这些时间段用于定位视频中的讲解顺序。严肃引用时，应回到视频片段确认讲者表述、板书或幻灯片内容。回看视频时，不必逐字抄录字幕。更有效的方法是把每个时间段判定为定义、例子、推导、代码解释或 caveat，再把它放回本章对应小节。

10.2 课程资料阅读路径

资料组	标题/页块	关键词
official_01	Lecture 11 S C A L I N G -C A S E S T U D Y A N D D E T A I L S CS336	scaling, scale, initialization, optimizers, cs336, motivation, best
official_02	Working through some detailed, public scaling recipes 1. MiniCPM 2. DeepSeek	scaling, minicpm, stabilize, many, working, detailed, public
official_03	Optimal LR According to muP –optimal learning rate should be (roughly) stable. Is it?	scaling, batch, learning, size, optimal, analysis, rate
official_05	Observation 3 (?) robustness Generalization to MoEs Generalization to other datasets	scaling, scale, update, parametrization, muon, gains, recall
official_06	What is muP not robust to? Exotic optimizers There are other, exotic optimizers based ...	scaling, robust, exotic, optimizers, strong, weight, decay

课程资料通常给出更稳定的标题、公式、图或代码结构；视频则补充动机和口头解释。两者合起来构成本章的事实边界。阅读资料时，可以把每个资料组拆成三个对象：一个定义，一个公式或伪代码动作，一个边界条件。这样比逐字翻译页面或脚本更容易形成可用知识。

11 实践说明、历史位置与风险

Scaling laws 进阶与实践在 CS336 的课程结构中连接基础机制和规模化问题。基础机制解释方法为什么成立；规模化问题检验它在更大模型、更长上下文、更复杂数据或真实 serving workload 下是否仍成立。历史位置不等于模型年表。更重要的是识别问题如何迁移：早期方法解决小规模建模问题，现代方法常常解决同一问题在大规模数据、GPU 集群、长上下文或人类偏好约束下的新形态。

- 资料版本限制 (version risk): 课程页、公开视频和配套资料可能在学期中更新；引用时应注明访问日期或资料版本。
- 测量口径限制 (measurement risk): FLOPs、tokens/s、benchmark score、reward 等数字只有在协议完整时可比较。
- 规模外推限制 (scaling risk): 小模型或小 batch 的机制不必然外推到 frontier-scale。
- 数据分布限制 (data risk): 数据来源、过滤和去重策略会改变模型行为，也会改变 evaluation contamination 风险。

12 复习要点

下面的问题把本章内容压缩为可反复检查的条目。每个条目都应能回到前文的解释、公式、伪代码或例题。

条目	对象	检查动作
条目 1	forecasting (预测)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 2	overtraining (过训练/超额训练)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 3	data-constrained scaling (数据受限缩放)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 4	muP	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 5	emergence (涌现)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。

12.1 综合自测

- 我能否不用课程原句，独立解释本章中心问题？
- 我能否把本章至少一个公式改写成可运行的估算函数？
- 我能否指出一个看似改进但实际转移成本的方法？
- 我能否回到视频和课程资料，找到支持本章关键论断的位置？
- 我能否设计一个最小 ablation，验证本章一个 caveat？

13 总结与延伸

13.1 本章小结

- Scaling laws 进阶与实践的核心不是记忆术语，而是理解对象、变量和机制之间的关系。
- 视频给出讲解顺序，课程资料提供可核对的公式、图、代码或页面结构。
- 复习时应把每个术语连接到至少一个变量、一个实现动作和一个失败模式。

13.2 延伸解释

本章中的延伸解释用于连接课程材料中的概念，例如说明公式里的 proxy、解释系统 tradeoff，或把多个材料片段串成一个完整推理链。若需要严肃引用，应回到课程视频和配套资料核对原始表述。

13.3 练习

- 定义题：用中英双语解释本章任意五个重要术语，并说明它们属于 compute、memory、data、optimization、evaluation 还是 deployment 账本。
- 推导题：选择本章一个公式，逐个解释符号、单位和近似假设，并说明哪个变量最难在真实系统中测量。
- 实现题：把本章伪代码改写成一个最小 Python sanity check，不要求训练大模型，但要输出可检查的中间量。
- 资料题：从“课程资料与回看路径”中选择一个视频时间段，回到原始视频核对讲者例子，并记录是否存在字幕误差。
- 评估题：为本章方法设计一个 ablation 或 benchmark，说明控制变量、数据版本、指标和 failure criteria。
- 边界题：说明本章一个结论在更长 context、更大 batch、更多 GPU 或更复杂数据分布下可能如何失效。
- 综合题：把本章内容和前一章或后一章连接起来，写出一个端到端训练/推理 pipeline 中的依赖关系。
- 批判题：指出一个容易被营销材料夸大的说法，并用本章的资源账本或评估协议反驳它。